TALK-TO-DATA (T2D)

Phase 3 Annexes: Governed Data Foundation

Daniel Brulé

LinkedIn - GitHub

Version 1.0 - June 2026

Table of Contents

1. How to use this annex pack

This annex pack contains example tools, templates and checklists supporting the **Phase 3 Governed Data Foundation Guide**. It is intended as practical inspiration for facilitation, documentation and delivery, not as a mandatory to-do list.

The main guide explains the delivery logic and decisions required during Phase 3. This annex provides reusable working material that teams can adapt depending on the delivery stage, business risk, data maturity, technology stack and governance requirements.

Teams should use only the templates that add value. For a narrow POC, many annex items may be simplified or skipped. For an MVP, pilot or production path, more of the material may be needed to support traceability, testing, security, cost control, ownership and operational handover.

2. Foundation tests, logs, cost and evidence controls

2.1. Testing controls

2.1.1. Testing principles

Principle	Meaning
Test before trust	Foundation assets should not be accepted because they build successfully; they should pass relevant tests.
Automate where possible	MVP, pilot and production foundations should rely on repeatable automated checks, not manual review alone.
Test by risk	High-risk metrics, joins, security rules and executive outputs need stronger tests than low-risk POC assets.
Keep evidence	Test results should be centrally stored and linked to the asset, metric, dimension or rule they validate.
Re-test on change	Changes to assets, metrics, joins, filters or security rules should trigger regression tests.

2.1.2. Test type definitions

Test type	Description	Where to use
Schema test	Confirms expected tables, views, columns and data types exist.	Queryable assets, deployment checks.
Completeness test	Checks that required fields are populated.	Assets, metrics, dimensions.
Uniqueness test	Confirms primary or business keys are unique where expected.	Dimension tables, lookup tables.
Referential integrity test	Confirms keys match valid records in another asset.	Fact-to-dimension relationships.
Grain test	Confirms the asset has the expected level of detail.	Curated assets, metric sources.
Join cardinality test	Checks joins do not duplicate or drop records unexpectedly.	Join and aggregation rules.
Metric calculation test	Validates implemented logic against the approved formula.	Metric implementation.
Filter / exclusion test	Confirms mandatory filters and exclusions are applied.	Standard filters and caveats.
Reconciliation test	Compares results against a trusted report or approved example.	Priority metrics and financial outputs.
Freshness test	Confirms data has refreshed within the expected window.	Assets with refresh dependencies.
Security policy test	Confirms row, column, masking and aggregation controls work.	Security and exposure controls.
Performance test	Measures query duration, refresh time or concurrency.	Pilot and production readiness.
Cost test	Checks whether queries, refreshes or tests stay within thresholds.	Scaling and run-cost control.
Regression test	Confirms existing outputs still work after changes.	Any foundation change.

2.1.3. Example testing technologies

Area	Example technologies
Data quality tests	dbt tests, Great Expectations, Soda, Deequ, SQL unit tests, platform-native data quality rules.
CI/CD test execution	GitHub Actions, GitLab CI, Azure DevOps, Jenkins, dbt Cloud jobs.
Orchestration-integrated tests	Airflow, Dagster, Prefect, Databricks Workflows, Azure Data Factory.
Warehouse-native checks	Snowflake tasks, BigQuery scheduled queries, Databricks SQL, Redshift/Synapse checks.
Test result centralisation	CI/CD dashboards, dbt artifacts, metadata tables, data catalogues, observability tools.

2.2. Logging controls

2.2.1. Logging principles

Principle	Meaning
Log what matters	Logs should support diagnosis, audit, cost control and operational support.
Link logs to assets	Logs should reference the relevant asset, metric, rule, version, environment or release.
Separate environments	Development, test, pilot and production logs should be distinguishable.
Alert by severity	Not every log needs escalation; critical failures and cost spikes should notify the right owner.
Define retention	Logs should be retained long enough to support audit, debugging and improvement.

2.2.2. Log type definitions

Log type	Description	Where to use
Build log	Records build status, timestamp, environment and version.	Deployment and CI/CD.
Deployment log	Records release, rollback, approval and deployment outcome.	Environment promotion.
Refresh log	Records refresh start/end time, duration, status and records processed.	Data pipelines and assets.
Quality log	Records passed tests, failed rules, thresholds and anomalies.	Data quality monitoring.
Change log	Records changes to metrics, dimensions, joins, filters or security rules.	Version and change control.
Access log	Records access attempts, permission changes or denied access.	Security validation and audit.
Query log	Records query frequency, duration, scanned data, failures and expensive queries.	Performance and cost control.
Alert log	Records triggered alerts, recipients, actions and resolution status.	Support and operations.
Incident log	Records production issues, impact, owner, resolution and follow-up.	Pilot and production operation.

2.2.3. Example logging technologies

Area	Example technologies
Cloud / platform logging	Azure Monitor, AWS CloudWatch, Google Cloud Logging.
Observability platforms	Datadog, Splunk, Elastic, Grafana, OpenTelemetry.
Warehouse query logs	Snowflake Query History, BigQuery Audit Logs, Databricks query history, Redshift system tables.
Orchestration logs	Airflow, Dagster, Prefect, Azure Data Factory, Databricks Workflows.
Security and audit logs	CloudTrail, Microsoft Purview, Google Cloud Audit Logs, SIEM tools.
CI/CD logs	GitHub Actions, GitLab CI, Azure DevOps, Jenkins.
Metadata / catalogue logs	DataHub, Collibra, Alation, OpenMetadata, Microsoft Purview.

2.3. Cost controls

2.3.1. Cost principles

Principle	Meaning
Report cost by layer	Avoid hiding foundation cost inside one platform total.
Separate build and run cost	Initial delivery effort is different from recurring refresh, query, test and monitoring cost.
Track useful units	Use cost per refresh, query, asset, metric, question or active user where helpful.
Identify cost drivers	Highlight expensive queries, high-frequency refreshes, large materialisations or excessive test runs.
Set thresholds and alerts	Cost spikes should trigger review before they become normalised.
Review before scaling	POC cost patterns should inform MVP, pilot and production sizing.

2.3.2. Cost layer definitions

Cost layer	Description	Example indicators
Storage	Cost of storing foundation assets and retained history.	Storage volume, growth rate, duplicate data.
Ingestion	Cost of extracting and moving source data.	API calls, batch runs, streaming volume.
Transformation	Cost of building models, views, tables and pipelines.	Compute time, job duration, retries.
Materialisation	Cost of precomputed tables, aggregates or cached outputs.	Storage and refresh cost of materialised assets.
Semantic / metadata	Cost of maintaining semantic definitions and metadata retrieval.	Catalogue refresh, embedding or indexing cost where relevant.
Query execution	Cost of user, assistant or evaluation queries.	Query count, scanned data, warehouse time.
Automated testing	Cost of CI/CD tests, reconciliation checks and regression runs.	Test runtime, compute consumed, test frequency.

Monitoring and logging	Cost of collecting and retaining logs, metrics and alerts.	Log volume, retention cost, observability platform cost.
Security and audit	Cost of policy enforcement, audit retention and reviews.	Audit log volume, access review effort.
Operations	Cost of support, remediation and change management.	Incidents, manual fixes, owner review time.

2.3.3. Example cost technologies

Area	Example technologies
Cloud cost management	Azure Cost Management, AWS Cost Explorer, GCP Billing.
Warehouse cost tracking	Snowflake cost views, BigQuery billing export, Databricks cost management, Redshift usage tables.
FinOps / optimisation tools	CloudHealth, Apptio, Kubecost, Infracost, native tagging and budget alerts.
Observability cost views	Datadog cost monitoring, Grafana dashboards, platform-native metrics.
Custom reporting	Cost metadata tables, tagged workloads, scheduled cost reports, BI dashboards.

3. Versioning and change control for foundation artefacts

Foundation artefacts should be versioned because T2D answers depend on the definitions, assets, rules and controls that were active when the answer was produced. Metrics, dimensions, joins, filters, caveats, security rules and cost thresholds may all change over time.

Versioning does not need to be heavy for a POC, but it should be explicit enough to understand what was used, what changed and who approved the change. For MVP, pilot or production, versioning should be linked to testing, deployment, ownership and rollback where appropriate.

Field	Description
Version	Current approved version of the asset, rule or definition.
Version status	Draft, approved, deprecated or retired.
Effective from	Date from which this version applies.
Effective to	Date until which this version applies, if known.
Last updated	Date of last approved change.
Change summary	Short description of what changed.
Change owner	Person or group accountable for approving the change.
Change route	How updates are reviewed, tested and released.

Versioning should apply to both the business definition and the technical implementation. For example, a metric definition may remain stable while the SQL view changes, or a join rule may change without the business label changing. The register should make clear which version was active when an answer, test result or release was produced.

For a POC, a simple version number and change note may be enough. For MVP or pilot, changes should normally be reviewed, tested and traceable through the agreed deployment route.

4. Recommended register technology by delivery stage

Stage	Recommended register technology	Why
POC	Governed spreadsheet, simple database table, or YAML/JSON files in Git	Fast, low-friction, enough to size MVP effort.
MVP	Versioned YAML/JSON, dbt metadata/semantic files, database metadata tables, or BI semantic-layer objects	More structured, testable, reviewable and easier to integrate.
Pilot	Metadata table + version-controlled definitions + catalogue/semantic-layer integration	Supports reuse, audit, testing and controlled change.
Production	Governed catalogue / semantic layer / metadata service integrated with CI/CD, lineage, access control and monitoring	Scalable, auditable and operationally maintainable.

Practical options

Option	Best use	Pros	Cons
Governed spreadsheet	Early POC, workshops, business review	Very fast, business-friendly	Weak automation, versioning and integration
YAML / JSON in Git	MVP-style engineering control	Versioned, reviewable, machine-readable, CI/CD friendly	Less friendly for business users without a UI
Database metadata tables	T2D needs to query the registry directly	Queryable, easy to integrate with orchestration	Needs update process and ownership
dbt metadata / semantic layer	dbt already used for transformations	Links definitions, tests, lineage and deployment	May not cover all business-facing context
BI semantic layer	Existing BI definitions are trusted	Reuses approved metrics and dimensions	Tool-specific; may be hard to expose safely
Data catalogue	Enterprise governance and ownership	Good for lineage, stewardship, discoverability	Often not structured enough for runtime T2D use
Metadata API/service	Mature production environment	Controlled, auditable, scalable	Higher build and operating effort

5. Activities

5.1. Confirm governed asset scope

5.1.1. Build scope summary

Item	Description
Delivery stage	POC / MVP / pilot / production path
Target users	User group in scope
Business domain	Domain covered by the foundation
Priority questions	Questions included in the current build
Main sources	Source systems, marts, models or reports used
Main exclusions	Questions, users, metrics or sources out of scope
Key caveats	Known limitations that must be surfaced or controlled
Decision owner	Person or group approving the build scope
Named owners	Accountable owners for metrics / data asset
operational contact	Day-to-day contacts for issue resolution
Version	Current version of the scope logic
Last update	Last update of the scope logic

5.1.2. Included question list

Priority question	Included?	Source asset	Metric / dimension dependency	Caveat	Owner
Example: Revenue by region last month	Yes	Sales mart	Net revenue, region, order date	Current month provisional	Finance analytics
Example: Margin by product	Deferred	Sales + cost models	Gross margin, product hierarchy	Cost allocation unresolved	Commercial finance

5.1.3. Scope decision categories

Category	Meaning
Include	Ready to implement in the current foundation.
Include with caveat	Can be implemented, but limitation must be documented and surfaced.
Remediate	Valuable, but requires data, logic, security or quality work first.
Defer	Potentially useful later, but not needed for the current stage.
Exclude	Not suitable for this T2D scope or delivery stage.

5.1.4. Final scope check

Before build starts, the team should be able to answer:

- What exactly is being implemented now?
- Which Phase 2-approved questions does it support?
- Which sources and metrics are excluded?
- What caveats must be visible to users or evaluators?

- Who owns each source, metric, caveat and exclusion?
- What new risks were discovered during build-scope confirmation?

5.2. Confirm implementation pattern and technology route

5.2.1. Delivery organisation and sprint setup

Area	Guidance
Delivery cadence	Use short delivery cycles with clear build, test and review checkpoints.
Squad shape	Include analytics engineering, data engineering, semantic ownership, security input, AI architecture and product ownership.
Sprint planning	Prioritise by question, metric or foundation asset rather than by technical component only.
Review points	Review implemented logic, access controls, caveats, quality checks and test evidence before accepting an asset.
Environment setup	Use code-based environments where possible so developers can build and test safely.
Definition of done	Asset is implemented, tested, documented, owned, versioned and ready for downstream build/evaluation.

5.2.2. Example technology routes

Route	Example
View-led foundation	Curated SQL views in Snowflake, BigQuery, Databricks or Redshift.
Transformation-led foundation	dbt or platform-native pipelines creating tested models.
Semantic-layer-led foundation	Looker, Cube, MetricFlow, Power BI semantic model or similar.
API-led foundation	Controlled service endpoints for approved metrics or sensitive queries.
Hybrid foundation	dbt models plus semantic-layer definitions plus materialised tables for high-volume queries.

5.2.3. Implementation patterns

Term	Definition
Curated views	Governed database views that expose approved columns, joins, filters or calculations for T2D use, without giving the assistant direct access to raw source tables.
Transformation models	Engineered data models, often built with tools such as dbt or platform-native pipelines, that transform raw or intermediate data into tested, documented and reusable analytical structures.
Semantic-layer objects	Governed business-facing definitions of metrics, dimensions, relationships, hierarchies and filters that allow the same business logic to be reused consistently across T2D, BI and analytics tools.
Materialised tables	Precomputed physical tables created to improve performance, reduce query cost or stabilise complex logic that would be too slow, expensive or fragile to calculate dynamically.

Controlled APIs	Approved service endpoints that expose specific data or calculations through managed logic, permissions and audit controls, rather than allowing open-ended SQL access.
Hybrid pattern	A combined foundation approach using more than one pattern, such as semantic-layer definitions over curated views, or APIs for sensitive calculations and materialised tables for high-volume metrics.

5.2.4. Pattern comparison

Pattern	Typical use	Strengths	Trade-offs
Curated views	Fast governed query surface	Simple, transparent, quick to build	Can become hard to manage at scale
Transformation models	Tested reusable data models	Versioned, documented, CI/CD-friendly	Requires engineering discipline
Semantic-layer objects	Consistent metrics and dimensions	Reusable across BI, analytics and T2D	Depends on tooling maturity and adoption
Materialised tables	Performance and cost control	Fast, stable, predictable queries	Adds storage and refresh management
Controlled APIs	Sensitive or constrained logic	Strong control, auditability and permissions	Less flexible for exploration
Hybrid pattern	Mixed enterprise environments	Pragmatic and adaptable	Needs clear ownership and standards

5.2.5. Technology decision checklist

Decision area	**Key question**
Storage platform	Where will approved assets physically live?
Transformation tool	How will models, views or tables be built?
Semantic layer	Where will metrics and dimensions be defined?
Access control	Where will row, column and masking rules be enforced?
Testing approach	How will logic, quality and regressions be tested?
Deployment approach	How will changes be reviewed and released?
Monitoring	How will freshness, failures and cost be tracked?
Ownership	Who owns the platform and foundation assets?

5.3. Curated queryable asset register

5.3.1. Asset register

Field	Description
Asset name	Name of the approved queryable asset.
Asset type	View, model, mart, semantic object, table or API.
Business purpose	What question or business need the asset supports.
Source dependency	Source system, table, model or upstream process used.
Supported questions	Priority questions the asset can support.
Key metrics	Metrics exposed through the asset.
Key dimensions	Dimensions available for filtering or grouping.
Grain	Lowest level of detail in the asset.
Refresh frequency	How often the asset is updated.
Data latency	Expected delay between source event and availability.
Owner	Accountable owner for the asset.
Operational contact	Day-to-day contact for issues or changes.
Access constraints	Row, column, masking or sensitivity restrictions.
Known caveats	Limitations that should be surfaced or considered.
Excluded fields	Fields deliberately not exposed to the assistant.
Maintenance route	How changes, fixes or questions are handled.
Upstream lineage	Main upstream sources, models or pipelines.
Data-layer position	Source, staging, curated, mart, semantic layer or medallion layer where applicable.
Version	Current version of the asset
Last update	Last update of the asset

5.3.2. Example register entry

Field	Example
Asset name	vw_t2d_sales_performance
Asset type	Curated view
Business purpose	Supports sales performance questions by region, product and month.
Source dependency	Sales mart, product dimension, region hierarchy.
Supported questions	Revenue by region; revenue by product category; monthly sales trend.
Key metrics	Net revenue, gross revenue, order count.
Key dimensions	Region, product category, month, customer segment.
Grain	One row per order line.
Refresh frequency	Daily.
Data latency	Previous day available by 08:00.
Owner	Commercial analytics owner.
Operational contact	Sales analytics engineering lead.
Access constraints	Regional row-level access applies.
Known caveats	Current month figures are provisional. Refunds may lag by 48 hours.
Excluded fields	Customer name, customer email, raw discount approval notes.

Maintenance route	Changes reviewed through analytics engineering backlog.

5.3.3. Asset exclusion log

Excluded asset / field	Reason for exclusion	Owner	Revisit condition
Raw customer table	Contains unnecessary personal data.	Customer data owner	Revisit if masked customer-level exploration is approved.
Discount approval notes	Free-text sensitive content.	Commercial operations	Revisit only after sensitivity review.
Legacy sales report	Definition conflicts with approved sales mart.	Finance analytics	Revisit if reconciliation is completed.

5.3.4. Final asset check

Before an asset is approved for T2D use, confirm:

- It supports at least one agreed priority question.
- Its grain, metrics and dimensions are understood.
- It exposes only the fields needed for the scoped use case.
- Access and sensitivity constraints are clear.
- Caveats and limitations are documented.
- Ownership and operational contacts are named.
- The maintenance route is known.

5.4. Metric implementation card

5.4.1. Metric implementation card

Field	Description
Metric name	Approved metric name used by the business.
Business definition	Plain-English definition of the metric.
Calculation logic	Formula or calculation rule used.
Source asset	Approved table, view, model, mart or API.
Grain	Lowest level at which the metric is calculated.
Date field	Date used for reporting and filtering.
Default time logic	Default period, comparison logic or time window.
Mandatory filters	Filters or exclusions always applied.
Allowed dimensions	Dimensions the metric can be grouped or filtered by.
Synonyms	Common user terms mapped to the metric.
Caveats	Limitations that should be surfaced where relevant.
Non-use cases	Questions or contexts where the metric should not be used.
Validation evidence	Reconciliation against trusted report or SME example.
Owner	Person or role accountable for the metric.
Operational contact	Day-to-day contact for issues or changes.
Lineage	Upstream sources, models or transformations.
Creation date	Date the implemented metric was created.
Version	Current version of the metric logic.
Change route	How updates are reviewed, tested and released.

5.4.2. Concrete implementation example

Field	Example
Metric name	Net revenue
Business definition	Revenue after discounts and refunds, excluding VAT and cancelled orders.
Calculation logic	`SUM(gross_revenue - discount_amount - refund_amount)`
Source asset	`vw_t2d_sales_order_line`
Grain	Order line
Date field	`order_date`
Default time logic	Calendar month based on `order_date`.
Mandatory filters	`order_status = 'completed';is_test_order = false; vat_amount excluded.`
Allowed dimensions	Month, region, product category, customer segment, sales channel.
Synonyms	Revenue, sales, net sales.
Caveats	Current month is provisional; refunds may lag by up to 48 hours.
Non-use cases	Do not use for statutory revenue reporting or audited financial statements.
Validation evidence	Reconciled to Commercial Sales Dashboard for March 2026 within agreed tolerance.
Owner	Finance analytics owner.
Operational contact	Sales analytics engineering lead.
Lineage	Sales order lines, refunds table, discount adjustments, product dimension, region hierarchy.
Creation date	2026-05-14
Version	v1.0
Change route	Changes reviewed by Finance Analytics and deployed through analytics engineering release process.

5.4.3. Example implementation logic

Below is a simplified example of how the metric might be implemented in a curated SQL view or transformation model.

```
SELECT
    order_line_id,
    order_id,
    order_date,
    region,
    product_category,
    customer_segment,
    sales_channel,
    gross_revenue,
    discount_amount,
    refund_amount,
    gross_revenue - discount_amount - refund_amount AS net_revenue
FROM sales_order_line
WHERE order_status = 'completed'
  AND is_test_order = false;
```

For T2D, the assistant should query the implemented `net_revenue` field or approved metric object. It should not recreate the calculation from memory or infer exclusions from the user's wording.

5.4.4. Metric validation check

Check	Example evidence
Calculation matches approved definition	Finance analytics sign-off.
Mandatory filters applied	Completed orders only; test orders excluded.
Date logic confirmed	Uses `order_date`, not invoice date or shipment date.
Grain confirmed	One row per order line.
Reconciles to trusted reference	March 2026 dashboard variance within agreed tolerance.
Caveats documented	Current month provisional; refund lag noted.
Owner and contact named	Finance owner and analytics engineering contact assigned.
Change route defined	Versioned release through analytics engineering process.

5.4.5. Metric registry

Metric registry structure

Field	Description
Metric ID	Stable unique identifier for the metric.
Approved metric name	Business-approved name exposed to users.
Business definition	Plain-English definition of the metric.
Implementation reference	Where the metric is implemented.
Implementation type	View, model, semantic layer, table, API or other.
Source asset	Approved asset used to calculate the metric.
Calculation logic	Formula or calculation rule.
Grain	Lowest level of detail used.
Date logic	Date field and default time behaviour.
Mandatory filters	Standard exclusions or filters always applied.
Allowed dimensions	Dimensions that can be used with the metric.
Synonyms	User terms mapped to the metric.
Caveats	Limitations that should be surfaced.
Non-use cases	When the metric should not be used.
Status	Approved, caveated, deferred, deprecated or excluded.
Owner	Accountable business or metric owner.
Operational contact	Day-to-day contact for questions or issues.
Validation evidence	Reconciliation or approval evidence.
Version	Current version of the metric definition.
Last updated	Date of last approved update.
Retrieval use	Whether the assistant can use this entry for routing, grounding or explanation.

Example metric registry entry

Field	Example
Metric ID	metric_net_revenue
Approved metric name	Net revenue
Business definition	Revenue after discounts and refunds, excluding VAT and cancelled orders.
Implementation reference	vw_t2d_sales_order_line.net_revenue
Implementation type	Curated view
Source asset	vw_t2d_sales_order_line
Calculation logic	SUM(gross_revenue - discount_amount - refund_amount)
Grain	Order line
Date logic	Uses order_date; default reporting period is calendar month.
Mandatory filters	Completed orders only; test orders excluded.
Allowed dimensions	Month, region, product category, customer segment, sales channel.
Synonyms	Revenue, sales, net sales.
Caveats	Current month is provisional; refunds may lag by 48 hours.
Non-use cases	Do not use for statutory revenue reporting.
Status	Approved for MVP
Owner	Finance analytics owner
Operational contact	Sales analytics engineering lead
Validation evidence	Reconciled to Commercial Sales Dashboard for March 2026 within agreed tolerance.
Version	v1.0
Last updated	2026-05-14
Retrieval use	Routing, grounding and answer explanation.

Possible implementation options

Implementation option	How it works	Strengths	Trade-offs
Governed spreadsheet	Metric registry maintained in a controlled spreadsheet.	Fast to start; accessible to business users; suitable for POC.	Weak versioning; harder to automate; can become unreliable at scale.
Data catalogue entry	Metrics documented in a catalogue or metadata management tool.	Good discoverability; supports ownership and lineage; familiar governance pattern.	May not be easy for the T2D system to query directly; depends on catalogue quality.
YAML / JSON files	Metric definitions stored as structured files in version control.	Versioned, reviewable, machine-readable; good for CI/CD and retrieval.	Less business-friendly unless supported by a UI or documentation layer.
dbt semantic layer / metrics layer	Metrics defined in dbt or similar transformation framework.	Strong link between definition, transformation, testing and lineage.	Requires engineering maturity; may not cover all business documentation needs.

BI semantic layer	Metrics defined in tools such as Looker, Power BI semantic model or similar.	Reuses existing BI definitions; aligns reporting and T2D.	Tool-specific constraints; may be hard to expose safely to the assistant.
Custom metadata table	Registry stored in a governed database table.	Queryable by T2D; easy to integrate with orchestration and retrieval.	Requires ownership, UI or process for updates.
API-backed registry	Metrics exposed through an approved metadata or metric API.	Strong control, auditability and integration.	More engineering effort; usually too heavy for early POC.

Recommended approach by delivery stage

Delivery stage	Recommended approach
POC	Governed spreadsheet, simple metadata table or YAML file with clear ownership.
MVP	Versioned YAML / JSON, metadata table, dbt metrics or BI semantic-layer integration.
Pilot	Registry linked to implemented logic, testing, ownership and change control.
Production	Governed, versioned, auditable registry integrated with catalogue, semantic layer, CI/CD and monitoring where possible.

Minimum registry checks

Before a metric is made available to T2D, confirm:

- The approved metric name and definition are clear.
- The implementation reference exists and is queryable.
- Mandatory filters, date logic and caveats are explicit.
- Allowed dimensions are defined.
- Synonyms are mapped or marked as ambiguous.
- The owner and operational contact are named.
- Validation evidence exists or the limitation is explicit.
- The metric status is clear: approved, caveated, deferred, deprecated or excluded.
- The entry is usable by the assistant for routing, grounding or explanation.

5.5. Dimension and hierarchy register

This annex provides a practical template for documenting the dimensions and hierarchies that the T2D system is allowed to use.

5.5.1. Dimension register structure

Field	Description
Dimension ID	Stable unique identifier for the dimension.
Approved label	Business-approved name exposed to users.
Business definition	Plain-English definition of the dimension.
Source asset	Approved table, view, model or semantic object.
Source field	Physical field or semantic object used.
Join key	Key used to connect the dimension to queryable assets.
Join path	Approved relationship between asset and dimension.
Grain	Lowest level of detail represented.
Hierarchy	Approved roll-up or drill-down structure.
Synonyms	User terms mapped to this dimension.
Valid metrics	Metrics this dimension can be used with.
Invalid metrics	Metrics this dimension should not be used with.
Caveats	Limitations or interpretation notes.
Non-use cases	Questions or contexts where it should not be used.
Owner	Accountable business or semantic owner.
Operational contact	Day-to-day contact for issues or changes.
Lineage	Upstream sources, models or transformations.
Creation date	Date the dimension entry was created.
Version	Current version of the dimension definition.
Last updated	Date of last approved update.
Change route	How updates are reviewed, tested and released.
Retrieval use	Whether the assistant can use this entry for routing, grounding or explanation.
Security implications	Access, masking or inference risks linked to this dimension.

5.5.2. Example dimension entry

Field	Example
Dimension ID	dim_region
Approved label	Region
Business definition	Commercial reporting region used for sales and performance analysis.
Source asset	dim_region
Source field	region_name
Join key	region_id
Join path	vw_t2d_sales_order_line.region_id = dim_region.region_id
Grain	Region
Hierarchy	Market > Country > Region
Synonyms	Market, territory, area
Valid metrics	Net revenue, gross revenue, order count, gross margin
Invalid metrics	Inventory value, statutory revenue
Caveats	Historic region mappings changed in 2024.

Non-use cases	Do not use for legal entity reporting.
Owner	Commercial operations
Operational contact	Sales analytics lead
Lineage	CRM region master, sales territory mapping, country reference table
Creation date	2026-05-14
Version	v1.0
Last updated	2026-05-14
Change route	Changes reviewed by Commercial Operations and deployed through analytics engineering release process.
Retrieval use	Routing, grounding and answer explanation.

5.5.3. Example implementation

Dimension table

```
CREATE TABLE dim_region (
  region_id STRING,
  region_name STRING,
  country_code STRING,
  country_name STRING,
  market_name STRING,
  sales_territory STRING,
  is_active BOOLEAN,
  valid_from DATE,
  valid_to DATE
);
```

Curated asset using the dimension

```
CREATE VIEW vw_t2d_sales_order_line AS
SELECT
  sol.order_line_id,
  sol.order_date,
  sol.region_id,
  r.region_name,
  r.country_name,
  r.market_name,
  r.sales_territory,
  sol.product_id,
  sol.customer_segment_id,
  sol.net_revenue
FROM sales_order_line sol
LEFT JOIN dim_region r
  ON sol.region_id = r.region_id
WHERE sol.order_status = 'completed'
 AND sol.is_test_order = false;
```

Semantic / metadata example

Yaml

```yaml
dimension_id: dim_region
approved_label: Region
description: Commercial reporting region used for sales analysis.
source_asset: dim_region
source_field: region_name
join_key: region_id
join_path: vw_t2d_sales_order_line.region_id = dim_region.region_id
hierarchy:
  - Market
  - Country
  - Region
synonyms:
  - market
  - territory
  - area
valid_metrics:
  - net_revenue
  - gross_revenue
  - order_count
  - gross_margin
invalid_metrics:
  - inventory_value
  - statutory_revenue
caveats:
  - Historic region mappings changed in 2024.
owner: Commercial operations
operational_contact: Sales analytics lead
status: approved_for_mvp
retrieval_use:
  - routing
  - grounding
  - answer_explanation
```

5.5.4. Compatibility matrix

Dimension	Valid metrics	Invalid metrics	Notes
Region	Net revenue, gross revenue, order count	Inventory value	Use commercial region hierarchy.
Product category	Net revenue, gross margin, order count	Customer churn	Product hierarchy maintained by Product Operations.
Customer segment	Net revenue, order count, churn	Statutory revenue	Segment assignment refreshed monthly.
Legal entity	Statutory revenue, cost, balance sheet metrics	Commercial net revenue	Use finance reporting hierarchy only.

5.5.5. Minimum dimension checks

Before a dimension is made available to T2D, confirm:

- The approved label and business definition are clear.
- The source asset and source field are known.
- The join key and join path are approved.
- The hierarchy is defined where relevant.
- Synonyms are mapped or marked as ambiguous.
- Valid and invalid metrics are defined.
- Caveats and non-use cases are documented.
- Owner and operational contact are named.
- The entry is versioned and has a change route.
- The assistant can use the entry for routing, grounding or explanation where appropriate.

5.6. Implement join, grain and aggregation register

5.6.1. Grain register

Field	Description
Asset name	Approved asset, view, model, table or semantic object.
Asset type	View, model, mart, semantic object, table or API.
Approved grain	Lowest level of detail in the asset.
Grain key	Field or combination of fields defining uniqueness.
Example row	Plain-English example of one row.
Valid metrics	Metrics that can be calculated from this grain.
Invalid metrics	Metrics that should not be calculated from this grain.
Owner	Accountable owner for grain definition.
Operational contact	Day-to-day contact for questions or issues.
Validation test	Test proving the grain is preserved.
Caveats	Known grain limitations or exceptions.
Version	Current version of the grain logic
Last update	Last update of the grain logic

Example

Field	Example
Asset name	vw_t2d_sales_order_line
Asset type	Curated view
Approved grain	Order line
Grain key	order_line_id
Example row	One row per product line in a customer order.
Valid metrics	Net revenue, gross revenue, order count, units sold.
Invalid metrics	Active customer count, customer churn.
Owner	Sales analytics owner
Operational contact	Analytics engineering lead
Validation test	order_line_id is unique and not null.
Caveats	Refunds may be updated after the original order date.

5.6.2. Approved join register

Field	Description
Join ID	Stable identifier for the join rule.
Left asset	Main asset being queried.
Right asset	Asset being joined.
Join key	Field or fields used for the join.
Relationship type	One-to-one, many-to-one, one-to-many or many-to-many.
Join status	Approved, restricted, excluded or under review.
Valid use	When this join may be used.
Invalid use	When this join should not be used.
Security implications	Access, masking or inference risks.
Required controls	Row rules, masking, aggregation threshold or clarification.
Owner	Accountable owner for the join rule.
Test evidence	Test proving join safety.
Version	Current version of the join
Last update	Last update of the join

Example

Field	Example
Join ID	join_sales_region
Left asset	vw_t2d_sales_order_line
Right asset	dim_region
Join key	region_id
Relationship type	Many-to-one
Join status	Approved
Valid use	Revenue, order count and margin by region, market or country.
Invalid use	Legal entity reporting.
Security implications	Region may drive row-level access.
Required controls	User region permission must be applied before aggregation.
Owner	Commercial operations
Test evidence	Join does not increase order-line row count.

5.6.3. Restricted or excluded join examples

Join	Status	Reason	Required action
Sales order line to customer profile	Restricted	Contains personal and sensitive customer attributes.	Only expose approved customer segment fields.
Sales order line to employee table	Restricted	May expose individual employee performance.	Aggregate above minimum group threshold.
Sales order line to raw discount notes	Excluded	Free-text field may contain sensitive information.	Do not expose to T2D.
Customer to multiple active segments	Under review	Many-to-many relationship can duplicate revenue.	Resolve segment rule or create bridge logic.

5.6.4. Aggregation rule register

Field	Description
Metric	Metric being aggregated.
Base grain	Level at which the metric is calculated.
Allowed aggregations	Sum, count, average, distinct count, min, max, etc.
Allowed dimensions	Dimensions the metric may be grouped by.
Disallowed dimensions	Dimensions that should not be used.
Required filters	Filters required before aggregation.
Minimum group size	Threshold below which results should be suppressed.
Caveats	Limitations to surface in the answer.
Validation evidence	Reconciliation or test result.
Version	Current version of the aggregation rule
Last update	Last update of the aggregation rule

Example

Field	Example
Metric	Net revenue
Base grain	Order line

Allowed aggregations	Sum
Allowed dimensions	Month, region, product category, customer segment.
Disallowed dimensions	Individual customer, individual employee.
Required filters	Completed orders only; test orders excluded.
Minimum group size	Suppress groups with fewer than 10 customers.
Caveats	Current month is provisional; refunds may lag by 48 hours.
Validation evidence	Monthly totals reconcile to Commercial Sales Dashboard.

5.6.5. Automated test examples

Test	Purpose	Example
Grain uniqueness test	Confirms the asset keeps the expected grain.	order_line_id is unique in vw_t2d_sales_order_line.
Join row-count test	Confirms the join does not duplicate rows.	Row count before and after joining dim_region is unchanged.
Orphan key test	Finds records with missing dimension matches.	Orders with region_id not found in dim_region.
Cardinality test	Confirms the expected relationship type.	Each region_id maps to one active region.
Aggregation reconciliation test	Confirms totals remain correct after grouping.	Revenue by region sums to total revenue.
Minimum group test	Confirms small groups are suppressed.	No result returned for groups below threshold.
Security join test	Confirms access rules survive joins.	User restricted to Region A cannot infer Region B after joining dimensions.

5.6.6. Minimum checks before approval

Before a join, grain or aggregation rule is approved for T2D use, confirm:

- The grain of each asset is explicit.
- The join path is approved and documented.
- The relationship type is known.
- Many-to-many joins are controlled or excluded.
- Aggregation behaviour is valid for the metric.
- Disallowed metric-dimension combinations are documented.
- Security and inference implications are identified.
- Automated tests exist where appropriate.
- Owner and operational contact are named.
- Caveats and limitations are available to the assistant where relevant.

5.7. Implement standard filter and caveat register

5.7.1. Filter register

Field	Description
Filter ID	Stable identifier for the filter.
Filter name	Business-friendly name of the filter.
Filter type	Mandatory, default, optional, clarification-required or excluded.
Business purpose	Why the filter is needed.
Technical logic	SQL condition, semantic rule or API parameter.
Applies to	Relevant asset, metric, dimension or question.
Default behaviour	Apply automatically, ask user, or only apply when requested.
User-facing wording	How the filter should be described to users.
Owner	Person or role accountable for the filter.
Operational contact	Day-to-day contact for issues or changes.
Validation evidence	Test, reconciliation or SME approval evidence.
Version	Current approved version.
Last updated	Date of last approved change.
Change route	How updates are reviewed, tested and released.

5.7.2. Example filter entry

Field	Example
Filter ID	filter_completed_orders
Filter name	Completed orders only
Filter type	Mandatory
Business purpose	Exclude cancelled, draft or failed transactions from sales reporting.
Technical logic	order_status = 'completed'
Applies to	Net revenue, gross revenue, order count, vw_t2d_sales_order_line
Default behaviour	Always apply automatically.
User-facing wording	“Only completed orders are included.”
Owner	Finance analytics owner
Operational contact	Sales analytics engineering lead
Validation evidence	Reconciled to Commercial Sales Dashboard.
Version	v1.0
Last updated	2026-05-14
Change route	Reviewed by Finance Analytics and deployed through analytics engineering release process.

5.7.3. Caveat register

Field	Description
Caveat ID	Stable identifier for the caveat.
Caveat name	Short business-friendly label.
Caveat type	Freshness, quality, definition, access, scope, reconciliation or provisional-period caveat.

Caveat wording	User-facing wording to surface where relevant.
Trigger condition	When the caveat should be shown.
Applies to	Relevant asset, metric, dimension, filter or question.
Severity	Low, medium or high.
Required behaviour	Show caveat, ask clarification, suppress answer or escalate.
Owner	Person or role accountable for the caveat.
Operational contact	Day-to-day contact for issues or changes.
Validation evidence	Evidence that caveat is correct and linked properly.
Version	Current approved version.
Last updated	Date of last approved change.
Change route	How updates are reviewed, tested and released.

5.7.4. Example caveat entry

Field	Example
Caveat ID	caveat_current_month_provisional
Caveat name	Current month provisional
Caveat type	Provisional-period caveat
Caveat wording	"Current month figures are provisional and may change after late adjustments."
Trigger condition	User asks for current-month sales, revenue or margin.
Applies to	Net revenue, gross revenue, margin, monthly sales trend
Severity	Medium
Required behaviour	Show caveat in the answer.
Owner	Finance analytics owner
Operational contact	Sales analytics engineering lead
Validation evidence	Finance confirms current-month adjustments remain open until month-end close.
Version	v1.0
Last updated	2026-05-14
Change route	Reviewed by Finance Analytics during month-end process updates.

5.7.5. Common clarification patterns

Clarification rules cannot be exhaustive. The aim is to define common ambiguity patterns and default behaviours, not to anticipate every possible user question. For MVP or pilot, the team should monitor real user questions, review ambiguous cases and update the filter, caveat and clarification rules over time.

User question	Issue	Assistant behaviour
"Show revenue for last period."	"Last period" is ambiguous.	Ask whether the user means last week, month, quarter or financial period.
"Show active customers."	Active customer definition may vary.	Use approved definition if available; otherwise ask clarification.
"Compare sales by region."	Region hierarchy may have multiple levels.	Use default approved hierarchy or ask if market, country or region is intended.

"Show revenue for this month."	Current month is provisional.	Answer with current-month caveat.
"Show customers with low revenue."	Potentially exposes customer-level detail.	Check security rules and aggregation thresholds before answering.

5.7.6. Automated test examples

Test	Purpose	Example
Mandatory filter test	Confirms required exclusions are applied.	Cancelled orders are excluded from net revenue.
Default filter test	Confirms default filters are applied consistently.	Current active products only are included unless user asks for historical product set.
Date logic test	Confirms the correct date field is used.	Sales reporting uses order_date, not invoice_date.
Caveat trigger test	Confirms caveats appear when conditions are met.	Current-month revenue triggers provisional-period caveat.
Clarification test	Confirms ambiguous terms trigger clarification.	"Last period" asks for period clarification.
Cross-asset consistency test	Confirms same rule is applied across assets.	Test orders are excluded from all sales metrics.
Regression test	Confirms filter behaviour does not change unexpectedly.	Existing priority questions still apply approved exclusions after release.

5.7.7. Minimum checks before approval

Before a filter or caveat is made available to T2D, confirm:

- The business purpose is clear.
- The technical logic is implemented or available to the assistant.
- The assets, metrics, dimensions or questions it applies to are known.
- Default behaviour is defined.
- User-facing wording is approved where needed.
- Clarification rules are defined for ambiguous cases.
- Validation evidence exists or the limitation is explicit.
- Owner and operational contact are named.
- Version and change route are defined.
- Automated tests exist where appropriate.

5.8. Security and exposure control register

5.8.1. User group and access matrix

Field	Description
User group / role	Approved user group, role or persona.
Source of group	Existing enterprise group, IAM role, HR hierarchy, territory mapping or manually defined group.
Linked master data	Master user, organisation, region, legal entity, cost centre or territory mapping used for access.
Approved assets	Assets the group may query.
Approved metrics	Metrics the group may use.
Approved dimensions	Dimensions the group may filter, group or drill down by.
Row-level scope	Rows the group is allowed to access.
Column restrictions	Columns hidden, restricted or unavailable.
Masking rules	Fields that must be masked or partially masked.
Aggregation limits	Minimum group-size, suppression or drill-down limits.
Owner	Person or team accountable for the access rule.
Operational contact	Day-to-day contact for issues or changes.
Version	Current approved version.
Last updated	Date of last approved change.

Example user group and access matrix

Field	Example
User group / role	Regional sales manager
Source of group	Enterprise IAM group
Linked master data	Sales territory master
Approved assets	vw_t2d_sales_order_line, dim_region, dim_product
Approved metrics	Net revenue, gross revenue, order count
Approved dimensions	Month, region, product category, customer segment
Row-level scope	User's assigned region only
Column restrictions	Customer email, customer name, discount approval notes excluded
Masking rules	Account ID partially masked
Aggregation limits	Suppress groups with fewer than 10 customers
Owner	Data governance lead
Operational contact	Analytics platform owner
Version	v1.0
Last updated	2026-05-14

5.8.2. Security control register

Field	Description
Control ID	Stable identifier for the security control.
Control name	Business-friendly name of the control.
Control type	Row-level, column-level, masking, aggregation, inference, audit or approval control.

Business purpose	Why the control is required.
Applies to	Relevant asset, field, metric, dimension, join, filter or user group.
User group / role	Users, roles or personas affected by the control.
Rule logic	Policy, SQL condition, semantic rule or platform control.
Default behaviour	Allow, restrict, mask, suppress, clarify or escalate.
Security owner	Person or team accountable for the control.
Operational contact	Day-to-day contact for issues or changes.
Approval route	How the control is approved or changed.
Audit requirement	What must be logged or retained.
Residual risk	Known risk that remains after the control is applied.
Version	Current approved version.
Effective from	Date from which the control applies.
Last updated	Date of last approved change.
Change route	How updates are reviewed, tested and released.

Example security control entries

Example 1 — Regional row-level access

Field	Example
Control ID	sec_region_rls
Control name	Regional row-level access
Control type	Row-level control
Business purpose	Users should only see sales data for regions they are authorised to access.
Applies to	vw_t2d_sales_order_line, Region dimension, Net revenue, Order count
User group / role	Regional sales managers
Rule logic	User region permission must match region_id before aggregation.
Default behaviour	Restrict rows before query result is returned.
Security owner	Data governance lead
Operational contact	Analytics platform owner
Approval route	Approved through data access governance process.
Audit requirement	Log role, region scope, query asset and denied access events.
Residual risk	Repeated cross-region comparisons must remain blocked.
Version	v1.0
Effective from	2026-05-14
Last updated	2026-05-14
Change route	Reviewed by data governance and deployed through platform access process.

Example 2 — Customer-level suppression

Field	Example
Control ID	sec_customer_group_threshold
Control name	Minimum customer group threshold
Control type	Aggregation / inference control
Business purpose	Prevent users from isolating very small customer groups.
Applies to	Customer segment, Region, Net revenue, Gross margin
User group / role	Commercial users

Rule logic	Suppress result where customer count is below 10.
Default behaviour	Suppress answer and explain that the group is too small.
Security owner	Privacy / data governance lead
Operational contact	Analytics engineering lead
Approval route	Privacy and governance review.
Audit requirement	Log suppressed-query event and applied threshold.
Residual risk	Repeated queries with different filters may still require monitoring.
Version	v1.0
Effective from	2026-05-14
Last updated	2026-05-14
Change route	Threshold changes reviewed by privacy, governance and data owner.

Exposure-risk examples

Exposure pattern	Example	Risk	Required response
Narrow drill-down	Revenue by store, customer segment and account manager	May isolate a small group or individual performance.	Apply minimum group threshold or suppress.
Sensitive dimension	Revenue by legal entity or employee location	May expose restricted finance or HR information.	Restrict dimension by role.
Masked field inference	Customer name hidden but account ID and postcode visible	User may infer customer identity.	Mask or remove indirect identifiers.
Unsafe join	Sales joined to employee table	May expose individual employee sales performance.	Restrict join or aggregate above threshold.
Repeated filters	User asks several overlapping questions to isolate one customer	Inference through differencing.	Monitor, suppress or escalate repeated-query pattern.
Free-text exposure	Discount notes or support comments exposed to T2D	May contain personal or commercially sensitive data.	Exclude free-text field unless reviewed and controlled.
Metadata leakage	Metric or caveat text reveals sensitive business context	User learns restricted information from retrieved metadata.	Classify and restrict metadata access.
Stale group mapping	User changes territory but old mapping remains active	User may see data for the wrong region.	Link to master user / territory view and refresh access mappings.

5.8.3. Security validation checks

Check	Purpose	Example
User group mapping test	Confirms users are assigned to the correct access group.	Regional manager maps to the correct current region.
Master-data alignment test	Confirms access mappings use the approved user, organisation or territory master.	User's region matches sales territory master.
Role-based access test	Confirms users only see permitted scope.	Regional manager can see Region A but not Region B.
Column restriction test	Confirms restricted fields are not exposed.	Customer email is unavailable to the assistant.
Masking test	Confirms sensitive values are masked correctly.	Account ID is masked for non-approved users.
Aggregation threshold test	Confirms small groups are suppressed.	No result returned for customer groups below threshold.
Join security test	Confirms access rules survive joins.	Joining region hierarchy does not expose unauthorised regions.
Metadata access test	Confirms restricted definitions or caveats are not retrieved by unauthorised users.	Restricted finance caveat is not shown to commercial users.
Repeated-query test	Checks whether users can infer sensitive values through follow-up questions.	Overlapping filters do not reveal single-customer values.
Audit log test	Confirms sensitive access events are logged.	Denied access and suppressed results are recorded.

5.8.4. Minimum checks before approval

Before a security or exposure control is approved for T2D use, confirm:

- User groups or roles are defined and linked to existing enterprise identity or master-data structures where possible.
- Sensitive assets, fields, dimensions, joins and metrics are identified.
- Row-level, column-level, masking and aggregation rules are implemented where required.
- Small-group and inference risks are considered.
- Security controls are tested with representative user roles.
- Denied, masked and suppressed outputs behave as expected.
- Audit and logging requirements are defined.
- Residual risks are documented and approved for the delivery stage.
- Owner, operational contact and change route are named.
- Version and effective date are recorded.
- Open security assumptions are handed over to security and governance validation.

5.9. Quality, performance, freshness, performance and cost controls

5.9.1. Quality control register

Field	Description
Control ID	Stable identifier for the quality control.
Control name	Business-friendly name of the check.
Control type	Completeness, uniqueness, validity, reconciliation, anomaly or threshold check.
Applies to	Relevant asset, metric, dimension, join, filter or question.
Rule logic	Test, SQL rule, data quality rule or platform check.
Expected threshold	Expected value, tolerance or pass/fail rule.
Severity	Low, medium or high.
Required behaviour	Warn, caveat, block, suppress or escalate.
Owner	Accountable owner for the control.
Operational contact	Day-to-day contact for issues.
Alert route	Who is notified when the control fails.
Validation evidence	Test result, reconciliation evidence or SME approval.
Version	Current approved version.
Last updated	Date of last approved change.
Change route	How updates are reviewed, tested and released.

Example quality control entry

Field	Example
Control ID	dq_sales_order_line_completeness
Control name	Sales order line completeness
Control type	Completeness check
Applies to	vw_t2d_sales_order_line
Rule logic	Required fields order_line_id, order_date, region_id, net_revenue must not be null.
Expected threshold	99.9% completeness for required fields.
Severity	High
Required behaviour	Block release if threshold fails; caveat existing environment if production refresh issue occurs.
Owner	Sales analytics owner
Operational contact	Analytics engineering lead
Alert route	Implementation team for dev/test; data owner and operating owner for pilot/production.
Validation evidence	Automated test results stored in central test log.
Version	v1.0
Last updated	2026-05-14
Change route	Reviewed through analytics engineering release process.

5.9.2. Freshness control register

Field	Description
Control ID	Stable identifier for the freshness control.
Asset / metric	Asset or metric subject to freshness expectations.
Refresh frequency	Expected refresh cadence.
Data latency	Expected delay between source event and availability.
Freshness threshold	Maximum acceptable delay before warning or blocking.
Failure behaviour	Warn, caveat, block, suppress or escalate.
User-facing caveat	Wording shown if freshness is degraded.
Owner	Accountable owner.
Operational contact	Day-to-day contact.
Alert route	Who is notified when freshness fails.
Version	Current approved version.
Last updated	Date of last approved change.

Example freshness control entry

Field	Example
Control ID	fresh_sales_daily
Asset / metric	vw_t2d_sales_order_line, Net revenue
Refresh frequency	Daily
Data latency	Previous day available by 08:00.
Freshness threshold	Warn after 09:00; block after 12:00 if prior-day data unavailable.
Failure behaviour	Show caveat after warning threshold; block current-day answer after block threshold.
User-facing caveat	"Sales data is delayed; latest confirmed refresh was yesterday at 08:00."
Owner	Sales analytics owner
Operational contact	Data engineering lead
Alert route	Data engineering team; operating owner if production threshold breached.
Version	v1.0
Last updated	2026-05-14

5.9.3. Performance control register

Field	Description
Control ID	Stable identifier for the performance control.
Query / asset / metric	Priority query, asset or metric covered.
Expected usage	POC, MVP, pilot or production usage expectation.
Performance target	Expected query runtime, refresh time or concurrency.
Threshold	Warning or failure threshold.
Measurement method	Query logs, warehouse metrics, orchestration logs or monitoring tool.
Required action	Optimise, materialise, cache, restrict, defer or escalate.
Owner	Accountable owner.
Operational contact	Day-to-day contact.
Version	Current approved version.
Last updated	Date of last approved change.

Example performance control entry

Field	Example
Control ID	perf_revenue_by_region
Query / asset / metric	Net revenue by region by month
Expected usage	MVP: up to 50 business users, repeated daily use.
Performance target	Query returns in under 10 seconds for standard filters.
Threshold	Warning above 10 seconds; optimisation required above 20 seconds.
Measurement method	Warehouse query logs and T2D orchestration logs.
Required action	Materialise monthly revenue aggregate if threshold repeatedly breached.
Owner	Analytics engineering lead
Operational contact	Data platform engineer
Version	v1.0
Last updated	2026-05-14

5.9.4. Cost reporting by meaningful layer

Cost reporting should separate infrastructure consumption from support effort. This avoids hiding the true cost of the governed foundation inside one platform total.

Cost layer	Cost type	What to track
Storage	Infrastructure	Storage volume, growth rate, retained history, duplicated data, materialised tables.
Ingestion	Infrastructure	Source extraction cost, data movement cost, API calls, batch frequency, streaming cost.
Transformation	Infrastructure	Compute used by transformations, model builds, incremental loads, backfills, failed or retried jobs.
Materialisation	Infrastructure	Storage and refresh cost of materialised tables, aggregates or cached outputs.
Query execution	Infrastructure	Query volume, query duration, scanned data, high-cost queries, concurrency, warehouse size.
Automated testing	Infrastructure	Cost of CI/CD tests, reconciliation checks, regression tests and validation jobs.
Monitoring and logging	Infrastructure	Log storage, metric collection, alerting, observability platform cost and retention cost.
Security and audit	Mixed	Audit log retention, policy enforcement overhead, access review effort.
T2D usage	Mixed	User questions, tool calls, generated queries, retries, failed queries, model/API costs where relevant.
Support and operations	Support	Incident handling, access requests, metric changes, caveat updates, quality remediation, owner review time.

5.9.5. Cost scenario template

Scenario	Usage assumption	Infrastructure cost drivers	Support cost drivers	Key risk
Narrow POC	Small user group, limited questions, limited refresh needs.	Low storage, limited compute, few test runs, minimal monitoring.	Light manual support, occasional clarification and metric review.	Underestimates cost of MVP controls.
Expected MVP	Target users, agreed priority questions, regular refresh and repeated usage.	Scheduled refresh, materialised assets, automated tests, query execution, monitoring.	Access requests, issue triage, metric updates, user feedback handling.	Costs grow if queries are inefficient or support model is unclear.
Higher-volume pilot	More users, more questions, stronger monitoring and security expectations.	Higher concurrency, larger refresh jobs, regression testing, log retention, cost alerts.	More incidents, change requests, governance reviews and operational support.	Foundation may need optimisation, caching or scope reduction.

5.9.6. Cost threshold register

Field	Description
Threshold ID	Stable identifier for the cost threshold.
Cost layer	Storage, ingestion, transformation, query execution, testing, monitoring, support, etc.
Metric	Cost indicator being tracked.
Expected range	Expected cost or consumption level.
Warning threshold	Level that triggers investigation.
Escalation threshold	Level that triggers owner review or delivery decision.
Alert recipient	Team or owner notified when threshold is breached.
Required action	Review, optimise, materialise, restrict, redesign or escalate.
Owner	Accountable owner.
Version	Current approved version.
Last updated	Date of last approved change.

Example cost threshold entry

Field	Example
Threshold ID	cost_query_scan_sales
Cost layer	Query execution
Metric	Data scanned per revenue query
Expected range	Under 5GB scanned per standard query.
Warning threshold	More than 10GB scanned.
Escalation threshold	More than 25GB scanned or repeated warning breaches.
Alert recipient	Implementation team in dev/test; product and operating owner in pilot/production.
Required action	Optimise filters, materialise aggregate or restrict unsupported breakdown.
Owner	Data platform owner
Version	v1.0
Last updated	2026-05-14

5.9.7. Automated control examples

Area	Example automated controls
Quality	Required fields not null, row counts within expected range, duplicate keys flagged, invalid values blocked.
Freshness	Alert when refresh exceeds threshold; caveat or block answers when data is stale.
Reconciliation	Compare metric totals with trusted dashboard or finance-approved reference.
Performance	Alert on slow priority queries, excessive scan volume or repeated query failures.
Cost	Alert on expensive queries, storage growth, high refresh cost, excessive test runs or monitoring cost spikes.
Support	Track recurring incidents, repeated access issues, unresolved quality failures and manual remediation effort.

5.9.8. Minimum checks before approval

Before quality, freshness, performance or cost controls are accepted, confirm:

- Critical assets, metrics and questions have defined quality checks.
- Freshness expectations and failure behaviour are clear.
- Reconciliation checks exist for priority metrics where relevant.
- Performance expectations are defined for priority queries.
- Cost is reported by meaningful layer.
- Infrastructure and support costs are separated.
- Cost scenarios exist for POC, MVP or pilot where needed.
- Alert thresholds and recipients are defined.
- Failed checks, cost anomalies and unresolved issues have owners.
- Version, review frequency and change route are recorded.

5.10. Governed foundation handover checklist

5.10.1. Governed foundation pack contents

Area	What to include
Foundation scope	Included questions, users, assets, metrics, dimensions and exclusions.
Queryable assets	Approved views, models, marts, semantic objects, materialised tables or APIs.
Metric logic	Metric implementation cards, registry entries, caveats and validation evidence.
Dimensions and hierarchies	Dimension register, hierarchy rules, synonyms and metric compatibility rules.
Joins and grain	Approved join paths, grain definitions, aggregation rules and excluded joins.
Filters and caveats	Standard exclusions, default filters, clarification rules and user-facing caveats.
Security controls	User groups, access rules, masking, aggregation thresholds and residual risks.
Quality and freshness	Data-quality checks, freshness thresholds, reconciliation checks and failure behaviour.
Performance and cost	Performance assumptions, cost scenarios, cost thresholds and cost observations.
Tests and logs	Centralised test results, deployment logs, validation logs and unresolved failures.
Ownership	Owners, operational contacts, approval routes and support responsibilities.
Change control	Versioning, release route, rollback route and communication process.
Backlog	Deferred items, remediation actions, known risks and future foundation improvements.

5.10.2. Handover checklist

Check	Status	Owner	Notes
Foundation scope is confirmed and documented.	Not started / In progress / Done		
Approved queryable assets are listed.	Not started / In progress / Done		
Asset lineage, creation date and ownership are recorded.	Not started / In progress / Done		
Metric implementation cards are complete for priority metrics.	Not started / In progress / Done		
Metric registry is versioned and linked to implementation.	Not started / In progress / Done		
Dimension register and hierarchy rules are complete.	Not started / In progress / Done		
Join, grain and aggregation rules are documented.	Not started / In progress / Done		

Standard filters, exclusions and caveats are documented.	Not started / In progress / Done		
Security controls and user-group mappings are documented.	Not started / In progress / Done		
Quality, freshness, performance and cost controls are documented.	Not started / In progress / Done		
Automated test results and validation evidence are centralised.	Not started / In progress / Done		
Logs, alerts and evidence locations are known.	Not started / In progress / Done		
Known limitations and residual risks are documented.	Not started / In progress / Done		
Support route and escalation path are agreed.	Not started / In progress / Done		
Change-control and release route are agreed.	Not started / In progress / Done		
Deferred items and remediation backlog are documented.	Not started / In progress / Done		

5.10.3. Downstream handover matrix

Receiving area	What they need from Phase 3
Architecture and orchestration	Approved assets, access patterns, metadata sources, query boundaries, performance assumptions and safe-failure needs.
Prototype / MVP build	Queryable assets, implemented metrics, dimensions, joins, filters, caveats and usage constraints.
Evaluation	Expected-answer sources, validation evidence, known caveats, failure cases and regression-test inputs.
Security validation	User groups, access controls, masking rules, aggregation thresholds, audit needs and residual risks.
Adoption	User-facing scope, supported questions, caveats, exclusions and escalation route.
Operations	Owners, contacts, logs, alerts, quality checks, cost tracking, support route and change-control process.

5.10.4. Known limitations and remediation backlog

Item	Type	Impact	Owner	Target action	Priority
Gross margin by product	Deferred metric	Cost allocation rule not approved.	Commercial finance	Confirm allocation rule and validation source.	High
Customer-level drill-down	Security limitation	Small-group inference risk.	Data governance	Define aggregation threshold and access rule.	High
Current-month revenue	Caveat	Figures may change after late adjustments.	Finance analytics	Surface provisional-period caveat.	Medium

Product hierarchy history	Data limitation	Historic comparisons may shift after hierarchy changes.	Product operations	Add hierarchy effective dating.	Medium

5.10.5. Support and escalation route

Issue type	First-line owner	Escalation owner	Example trigger
Asset refresh failure	Data engineering	Operating owner	Daily sales asset not refreshed by agreed threshold.
Metric definition question	Metric owner	Product owner / domain owner	User challenges revenue definition.
Data-quality failure	Data owner	Governance lead	Reconciliation gap exceeds tolerance.
Access issue	Security / platform owner	Governance lead	User sees too much or too little data.
Caveat or explanation issue	Semantic owner	Product owner	Assistant does not surface required caveat.
Cost spike	Platform owner	Product owner / finance owner	Query or refresh cost exceeds threshold.
User feedback	Product owner	Business sponsor	Repeated confusion or unsupported questions.

5.10.6. Final handover decision

Question	Answer
Is the governed foundation fit for the intended next step?	Yes / No / Yes with caveats
What delivery stage is it fit for?	POC / MVP / pilot / production path
What is not yet fit for broader use?	
What risks remain?	
What must be remediated before scaling?	
Who owns the next decision?	
Date of handover decision	

This final decision should be short and explicit. The goal is not to prove that the full T2D product is ready, but to confirm whether the governed foundation is controlled, understood and usable for the next delivery stage.